Pros and cons of propositional logic

☺ Propositional logic is **declarative**: pieces of syntax correspond to facts

☺ Propositional logic allows partial/disjunctive/negated information
(unlike most data structures and databases)

☺ Propositional logic is **compositional**:
meaning of $B_{1,1} \wedge P_{1,2}$ is derived from meaning of $B_{1,1}$ and of $P_{1,2}$

☺ Meaning in propositional logic is **context-independent**
(unlike natural language, where meaning depends on context)

☹ Propositional logic has very limited expressive power
(unlike natural language)
E.g., cannot say "pits cause breezes in adjacent squares"
except by writing one sentence for each square

First-order logic:
- √ Extends propositional logic
- √ Keeps good features
- √ Adds expressiveness

Logics in general

Language	Ontological Commitment	Epistemological Commitment
Propositional logic	facts	true/false/unknown
First-order logic	facts, objects, relations	true/false/unknown
Temporal logic	facts, objects, relations, times	true/false/unknown
Probability theory	facts	degree of belief
Fuzzy logic	facts + degree of truth	known interval value

- *Ontological* commitment
  - Ontology - in philosophy, the study of "what is"
  - What are the underlying assumptions of the logic with respect to the nature of *reality*
- *Epistemological* commitment
  - Epistemology - in philosophy, the study of "what can be known"
  - What are the underlying assumptions of the logic with respect to the nature of what the agent can *know*

Wumpus

- Squares neighboring the wumpus are smelly
  - Objects: Wumpus, squares
  - Property: Smelly
  - Relation: neighboring
- Evil king john rules England in 1200
  - Objects: John, England, 1200
  - Property: evil, king
  - Relation: ruled

First-Order Logic

- Idea:
  - Don't treat propositions as "atomic" entities.
- First-Order Logic:
  - Objects: cs4701, fred, ph219, emptylist …
  - Relations/Predicates: is_Man(fred), Located(cs4701, ph219), is_kind_of(apple, fruit)…
    - Note: Relations typically correspond to verbs
  - Functions: Best_friend(), beginning_of() : Returns object(s)
  - Connectives: $\wedge$, $\vee$, $\neg$, $\Rightarrow$, $\Leftrightarrow$
  - Quantifiers:
    - Universal: $\forall$x: (is_Man(x)) is_Mortal(x))
    - Existential: $\exists$y: (is_Father(y, fred))

- First-order logic is based on "common sense" or linguistic concepts:
- First-order logic is the most important and best understood logic in philosophy, mathematics, and AI

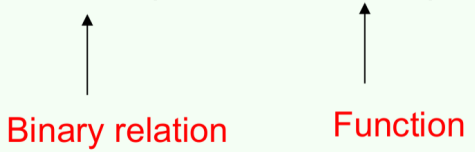
Atomic sentences

Atomic sentence $= predicate(term_1, \ldots, term_n)$
or $term_1 = term_2$

Term $= function(term_1, \ldots, term_n)$
or $constant$ or $variable$

E.g., $Brother(KingJohn, RichardTheLionheart)$
$> (Length(LeftLegOf(Richard)), Length(LeftLegOf(KingJohn)))$

- Sentences in logic state facts that are true or false.
- Properties and n-ary relations do just that:
  LargerThan(2,3) (means 2>3) is false.
  Brother(Mary,Pete) is false.
- Note: Functions do not state facts and form no sentence:
  Brother(Pete) refers to the object John (his brother) and
  is neither true nor false.
- Brother(Pete,Brother(Pete)) is True.



Complex sentences

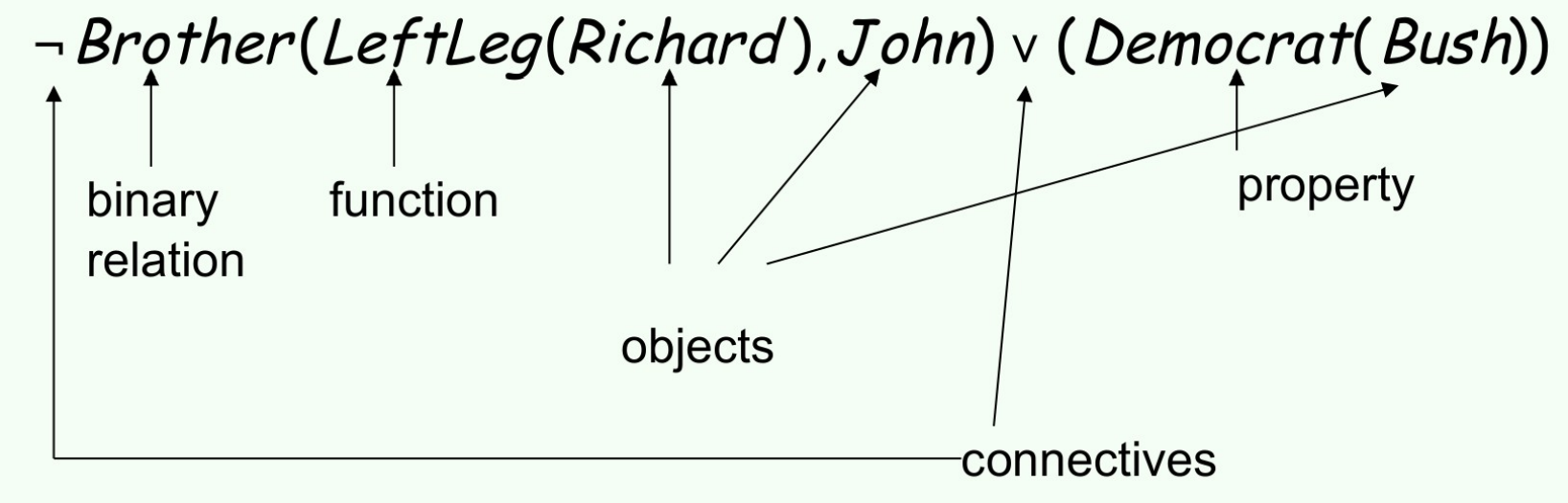
Complex sentences are made from atomic sentences using connectives

$$\neg S, \quad S_1 \wedge S_2, \quad S_1 \vee S_2, \quad S_1 \Rightarrow S_2, \quad S_1 \Leftrightarrow S_2$$

E.g. $Sibling(KingJohn, Richard) \Rightarrow Sibling(Richard, KingJohn)$

$>(1,2) \vee \leq(1,2)$

$>(1,2) \wedge \neg>(1,2)$



Truth in first-order logic

Sentences are true with respect to a model and an interpretation

Model contains ≥ 1 objects (domain elements) and relations among them

Interpretation specifies referents for

- constant symbols $\rightarrow$ objects
- predicate symbols $\rightarrow$ relations
- function symbols $\rightarrow$ functional relations

An atomic sentence $predicate(term_1, \ldots, term_n)$ is true
iff the objects referred to by $term_1, \ldots, term_n$
are in the relation referred to by $predicate$

Truth example

Consider the interpretation in which
$Richard \longrightarrow$ Richard the Lionheart
$John \longrightarrow$ the evil King John
$Brother \longrightarrow$ the brotherhood relation

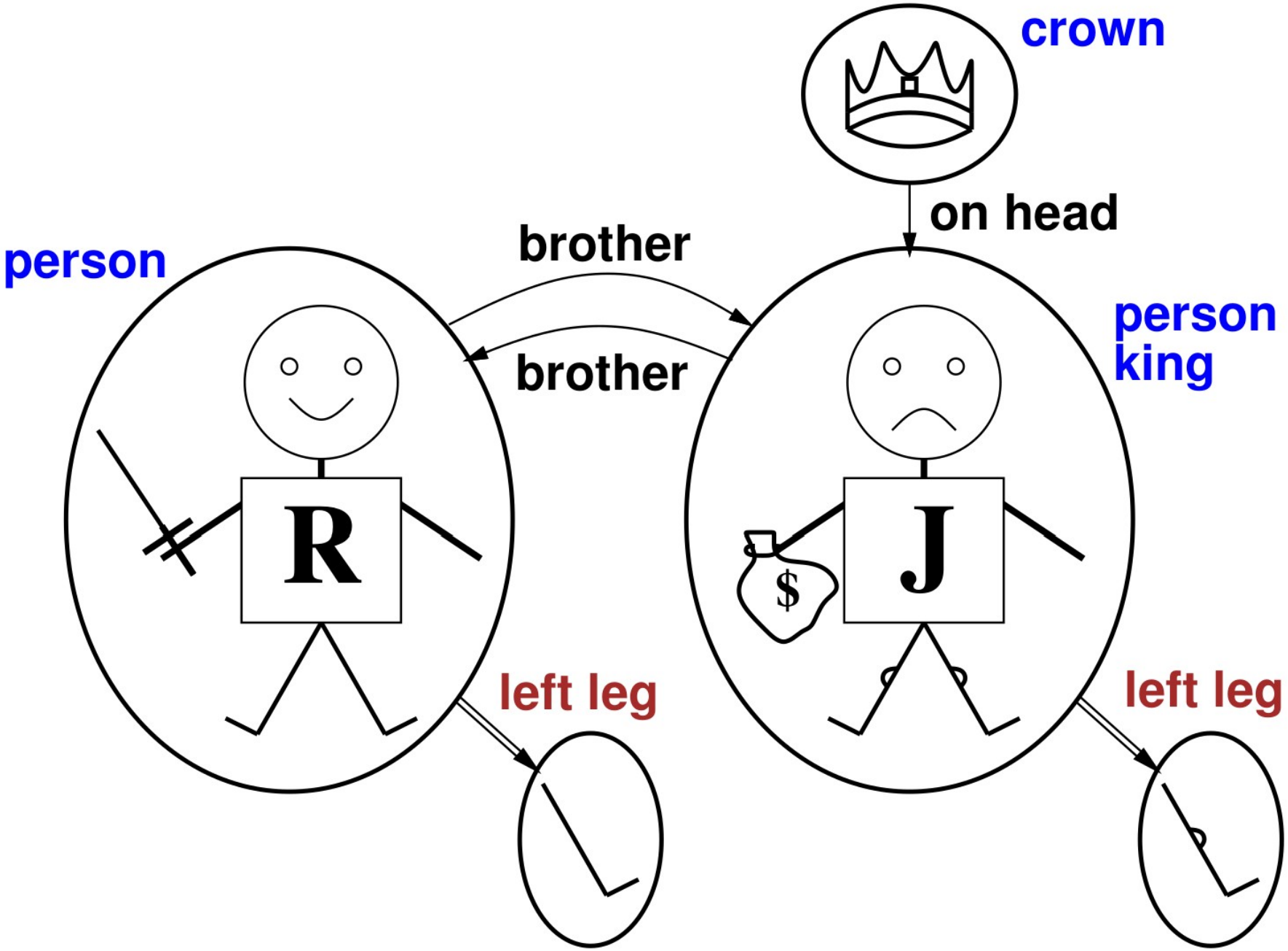
Under this interpretation, $Brother(Richard, John)$ is true just in case Richard the Lionheart and the evil King John are in the brotherhood relation in the model

Variables

- Person(John) is true or false because we give it a single argument ‘John’

- We can be much more flexible if we allow variables which can take on values in a domain. e.g., all persons x, all integers i, etc.
  - E.g., can state rules like Person(x) => HasHead(x)
    or Integer(i) => Integer(plus(i,1)

Models for FOL: Example



Quantifiers and equality

- Quantifiers are used to express properties of *classes of objects*
- *Universal* quantification $\forall$
  - √ "For all .." quantification
  - √ Example: $\forall x\ Cat(x) \Rightarrow Mammal(x)$
- *Existential* quantification $\exists$
  - √ "There exists .." quantification
  - √ Example: $\exists x\ Sister(x, Spot) \wedge Cat(x)$
- Equality =
  - √ Two terms refer to same object
  - √ Example: *Father(John)* = *Henry*

Universal quantification

$\forall \langle variables \rangle \ \langle sentence \rangle$

Everyone at Berkeley is smart:
$\forall x \ \ At(x, Berkeley) \Rightarrow Smart(x)$

$\forall x \ \ P$ is true in a model m iff P is true with x being **each** possible object in the model

Roughly speaking, equivalent to the conjunction of instantiations of P

$$
\begin{aligned}
& (At(KingJohn, Berkeley) \Rightarrow Smart(KingJohn)) \\
\wedge \ & (At(Richard, Berkeley) \Rightarrow Smart(Richard)) \\
\wedge \ & (At(Berkeley, Berkeley) \Rightarrow Smart(Berkeley)) \\
\wedge \ & \ldots
\end{aligned}
$$

Existential quantification

$\exists \langle variables \rangle \ \langle sentence \rangle$

Someone at Stanford is smart:
$\exists x \ \ At(x, Stanford) \wedge Smart(x)$

$\exists x \ \ P$ is true in a model m iff P is true with x being **some** possible object in the model

Roughly speaking, equivalent to the disjunction of instantiations of P

$$\begin{aligned} & (At(KingJohn, Stanford) \wedge Smart(KingJohn)) \\ \vee \ & (At(Richard, Stanford) \wedge Smart(Richard)) \\ \vee \ & (At(Stanford, Stanford) \wedge Smart(Stanford)) \\ \vee \ & \ldots \end{aligned}$$

A common mistake to avoid

Typically, $\Rightarrow$ is the main connective with $\forall$

Common mistake: using $\wedge$ as the main connective with $\forall$:

$$\forall x \;\; At(x, Berkeley) \wedge Smart(x)$$

means "Everyone is at Berkeley and everyone is smart"

Another common mistake to avoid

Typically, $\wedge$ is the main connective with $\exists$

Common mistake: using $\Rightarrow$ as the main connective with $\exists$:

$$\exists x \;\; At(x, Stanford) \Rightarrow Smart(x)$$

is true if there is anyone who is not at Stanford!

Common mistakes to avoid

- $\Rightarrow$ is the main connective with $\forall$
- $\wedge$ is the main connective with $\exists$

$\forall x, King(x) \Rightarrow Person(x) \quad x = \{Pete, Mary, tablespoon\}$

$\forall x, King(x) \wedge Person(x)$

$\exists x, King(x) \Rightarrow Person(x)$

$\exists x, King(x) \wedge Person(x)$

All of these must be true!
King(Pete) AND Person(Pete)
King(Mary) AND Person(Mary)
King(Tablespoon) AND Person(Tablespoon)

too strong

False!

One of these should be true!
if King(Pete) then Person(Pete)
if King(Mary) then Person(Mary)
If King(Tablespoon) then Person(Tablespoon) True!

too weak

Properties of quantifiers

Nested Quantifiers

$\forall x \ \forall y$ is the same as $\forall y \ \forall x$ (why??)

$\exists x \ \exists y$ is the same as $\exists y \ \exists x$ (why??)

$\exists x \ \forall y$ is **not** the same as $\forall y \ \exists x$

$\exists x \ \forall y \ Loves(x, y)$

"There is a person who loves everyone in the world"

$\forall y \ \exists x \ Loves(x, y)$

"Everyone in the world is loved by at least one person"

Quantifier duality: each can be expressed using the other

$\forall x \ Likes(x, IceCream)$ $\quad\quad$ $\neg\exists x \ \neg Likes(x, IceCream)$

$\exists x \ Likes(x, Broccoli)$ $\quad\quad$ $\neg\forall x \ \neg Likes(x, Broccoli)$

De Morgan's Law for Quantifiers

De Morgan's Rule

$$P \wedge Q \equiv \neg(\neg P \vee \neg Q)$$

$$P \vee Q \equiv \neg(\neg P \wedge \neg Q)$$

$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$$

$$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$$

Generalized De Morgan's Rule

$$\forall x\, P \equiv \neg \exists x\, (\neg P)$$

$$\exists x\, P \equiv \neg \forall x\, (\neg P)$$

$$\neg \forall x\, P \equiv \exists x\, (\neg P)$$

$$\neg \exists x\, P \equiv \forall x\, (\neg P)$$

Rule is simple: if you bring a negation inside a disjunction or a conjunction, always switch between them (or →and, and → or).

- Equality symbol: Father(John)=Henry.
  This relates two objects.

Fun with sentences

Brothers are siblings

$$\forall x, y \;\; Brother(x, y) \;\Rightarrow\; Sibling(x, y).$$

"Sibling" is symmetric

$$\forall x, y \;\; Sibling(x, y) \;\Leftrightarrow\; Sibling(y, x).$$

One's mother is one's female parent

$$\forall x, y \;\; Mother(x, y) \;\Leftrightarrow\; (Female(x) \wedge Parent(x, y)).$$

A first cousin is a child of a parent's sibling

$$\forall x, y \;\; FirstCousin(x, y) \;\Leftrightarrow\; \exists p, ps \;\; Parent(p, x) \wedge Sibling(ps, p) \wedge Parent(ps, y)$$

Translating English to FOL

- Every gardener likes the sun.
  `(Ax) gardener(x) => likes(x,Sun)`

- You can fool some of the people all of the time.
  `(Ex)(At) (person(x) ^ time(t)) => can-fool(x,t)`

- You can fool all of the people some of the time.
  `(Ax)(Et) (person(x) ^ time(t) => can-fool(x,t)`

- All purple mushrooms are poisonous.
  `(Ax) (mushroom(x) ^ purple(x)) => poisonous(x)`

- No purple mushroom is poisonous.
  `~(Ex) purple(x) ^ mushroom(x) ^ poisonous(x)`
  or, equivalently,
  `(Ax) (mushroom(x) ^ purple(x)) => ~poisonous(x)`

- There are exactly two purple mushrooms.
  `(Ex)(Ey) mushroom(x) ^ purple(x) ^ mushroom(y) ^ purple(y) ^ ~(x=y) ^ (Az) (mushroom(z) ^ purple(z)) => ((x=z) v (y=z))`

Using first-order logic: Kinship domain

- The *domain* of family relationship
  - √ Objects: *Persons*
  - √ Unary predicates: *Male*, *Female*
  - √ Binary predicates: *Parent*, *Sibling*, *Brother*, *Sister*, …
  - √ Functions: *Mother*, *Father*
- Example sentences

$$\forall m,c\ Mother(c) = m \Leftrightarrow Female(m) \wedge Parent(m,c)$$

$$\forall w,h\ Husband(h,w) \Leftrightarrow Male(h) \wedge Spouse(h,w)$$

$$\forall g,c\ Grandparent(g,c) \Leftrightarrow \exists p\ Parent(g,p) \wedge Parent(p,c)$$

$$\forall x,y\ Sibling(x,y) \Leftrightarrow x \neq y \wedge \exists p\ Parent(p,x) \wedge Parent(p,y)$$

TELLing and ASKing sentences

- TELL an agent about kinship using *assertions*
  - √ TELL(KB, $(\forall m,c\ Mother(c) = m \Leftrightarrow Female(m) \wedge Parent(m,c)$))
  - √ TELL(KB, $(Female(Maxi) \wedge Parent(Maxi, Spot) \wedge Parent(Spot, Boots)$))
- ASK an agent by posing *queries* (or *goals*)
  - √ ASK(*KB*,($Grandparent(Maxi, Boots)$))
    - Answer: *Yes*
  - √ ASK(*KB*, ($\exists x\ Child(x, Spot)$))
    - Answer: *x = Boots*

Wumpus world revisited

- *Properties* of locations

  $\forall x, t \;\; At(Agent,x,t) \wedge Breeze(t) \Rightarrow Breezy(x)$

- *Diagnostic* rules – infer cause from effect

  $\forall y \; Breezy(y) \Rightarrow \exists x \; Pit(x) \wedge Adjacent(x,y)$

- *Causal* rules – infer effect from cause

  $\forall x,y \; Pit(x) \wedge Adjacent(x,y) \Rightarrow Breezy(y)$

- *Definition* of predicate

  $\forall y \; Breezy(y) \Leftrightarrow (\exists x \; Pit(x) \wedge Adjacent(x,y))$

Inference in first-order logic

- Reduce to propositional inference
- Generalized Modus Ponens
- Forward chaining
- Backward chaining
- Resolution

- Require *substitution* and/or *unification* of terms

Substitution and unification

- *Substitution* is the replacement of variable(s) in a sentence with expressions
  - √ SUBST({*x*/*Richard*, *y*/*John*}, *Brother*(*x*,*y*)) = *Brother*(*Richard*, *John*)
- *Unification* is finding substitutions (a *unifier*) that make different sentences look identical
  - √ UNIFY(*p*,*q*) = *Θ* where SUBST(*Θ*,*p*) = SUBST(*Θ*,*p*)
  - √ E.g.

$$UNIFY(Knows(John, x), Knows(John, Jane)) = \{x / Jane\}$$

$$UNIFY(Knows(John, x), Knows(y / Bill)) = \{x / Bill, y / John\}$$

$$UNIFY(Knows(John, x), Knows(y, Mother(y))) = \{y / John, x / Mother(John)\}$$

$$UNIFY(Knows(John, x), Knows(x, Elizabeth)) = fail$$

Example knowledge base

- Known facts
  - √ It is a crime for an American to sell weapons to hostile nations. The country Nono is an enemy of America and has some missiles, all sold to it by Colonel West, an America.
- Knowledge base:

$$American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Cri\min al(x)$$

$$Owns(Nono, M1), Missile(M1), Missile(x) \Rightarrow Weapon(x)$$

$$Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(West, x, Nono)$$

$$Enemy(x, America) \Rightarrow Hostile(x)$$

$$American(West), Enemy(Nono, America)$$

- Prove that West is a criminal

Reduce to propositional inference

- Can reduce first-order sentence to propositional sentence by instantiation rules
  - *UI - Universal instantiation* - Substitutes a constant in KB for a universally quantified variable
  - *EI - Existential instantiation* - Substitutes a *new* constant for an existentially quantified variable
- Apply UI and EI systematically to replace all sentences with quantifiers with variable-free sentences
- Can the propositional inference rules to derive proofs, but it is an inefficient procedure

Generalized modus ponens - GMP

- Assumes knowledge base containing facts and rules like

$$p_1 \wedge p_2 \wedge \Lambda \wedge p_n \Rightarrow q$$

- The *generalized modus ponens* rule

$$\frac{p'_1, p'_2, \Lambda\ , p'_n, (p_1 \wedge p_2 \wedge \Lambda \wedge p_n \Rightarrow q)}{SUBST(\theta, q)}$$

$$SUBST(\theta, p'_i) = SUBST(\theta, p_i) \quad for\ all\ i$$

- Example

$$Missile(M1)$$

$$SUBST = \{x / M1\}$$

$$Owns(Nono, M1)$$

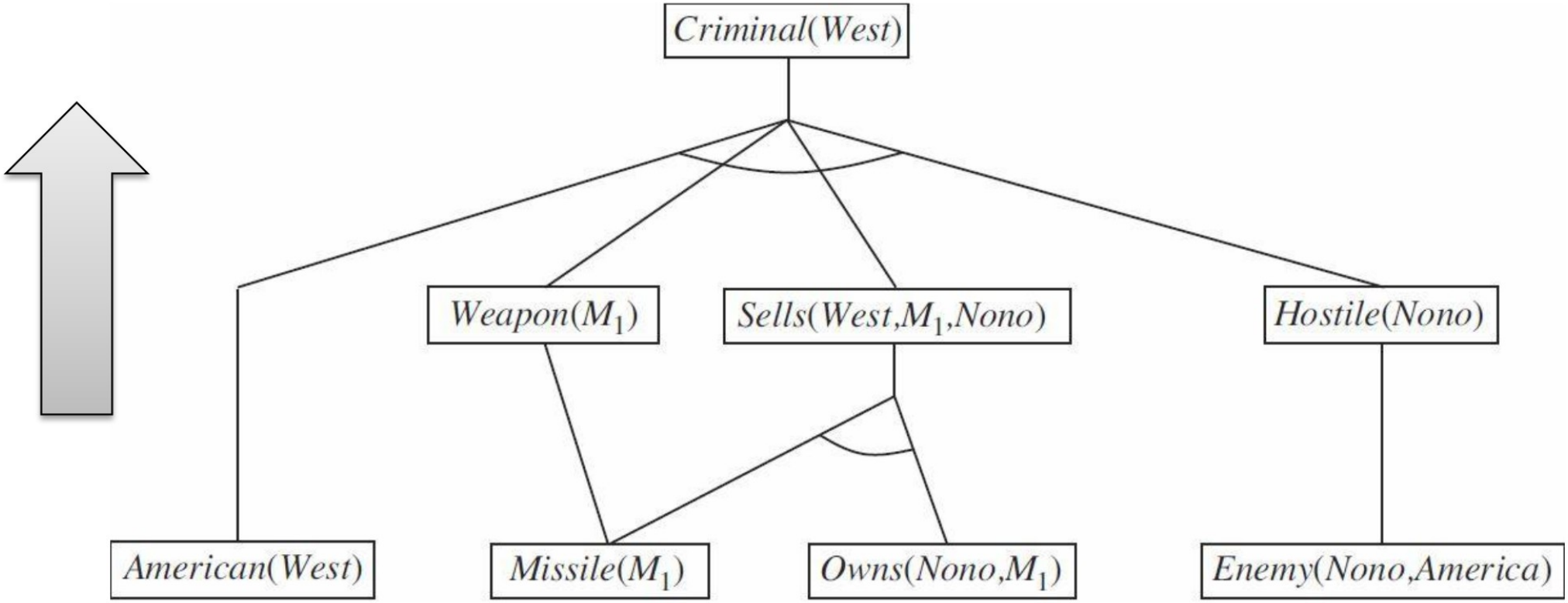
$$\frac{\forall x\ Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(West, Nono, x)}{Sells(West, Nono, M1)}$$

Completeness of first-order logic

- First-order logic is *complete*
  - If a sentence is entailed by KB, then this can be proved
  - (Gödel's completeness theorem, 1930)
  - But generalized modus ponens is *not* complete
- However, first-order logic is *semi-decidable*
  - If the sentence is *not* entailed by the KB, this can not always be shown
- Extension of first-order logic with mathematical induction is incomplete
  - There are true statements that cannot be proved
  - (Gödel's incompleteness theorem, 1931)

Forward chaining - FC

- Start with sentences in KB, apply inference rules in forward direction, adding new sentences until goal found or no further inference can be made

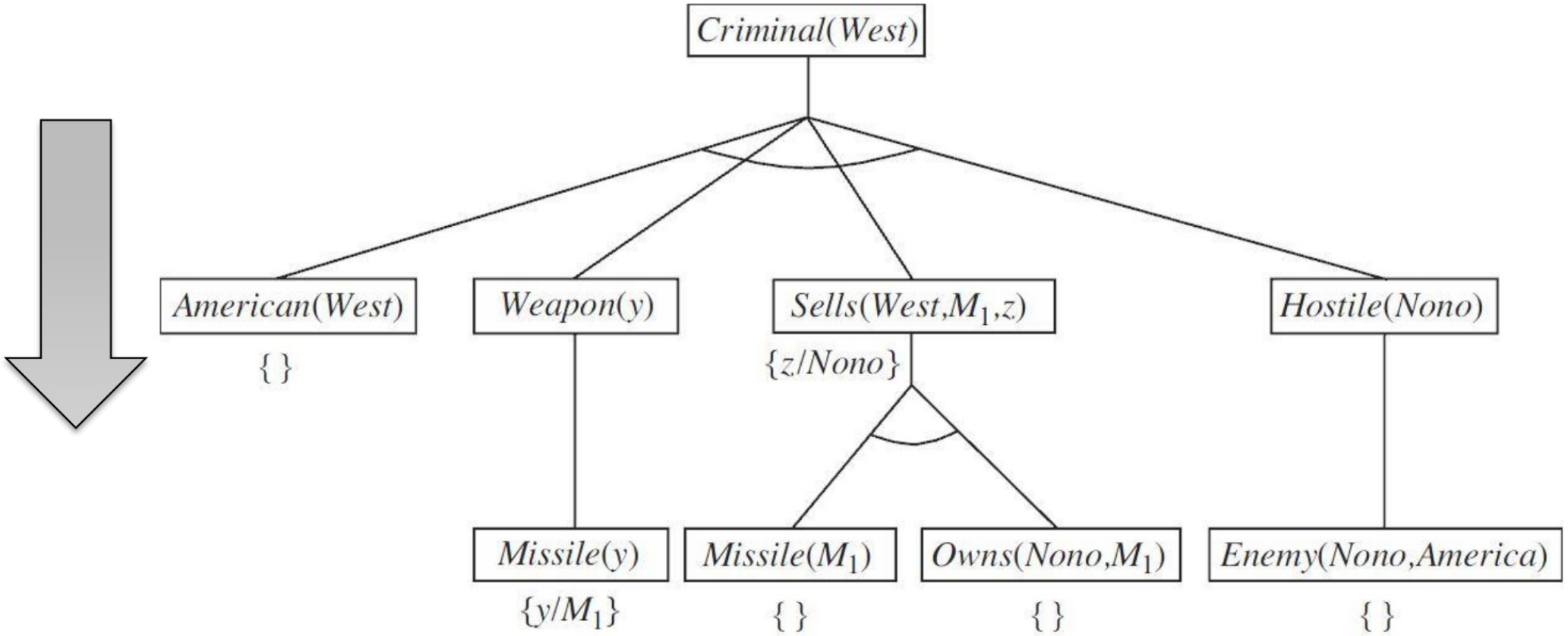


Production systems using FC

- Main features
  - √ Consists of rule base and working memory
  - √ Uses rule matching and forward chaining to add facts to the working memory until a solution is found
  - √ *Rete networks* are used to speed up matching
- Applications
  - √ Used in many expert systems, especially early ones
  - √ Design and configuration systems
  - √ Real-time monitoring and alarming
  - √ "Cognitive architectures" (SOAR)

Backward chaining - BC

- Start with goal sentence, search for rules that support goal, adding new sub-goals until match with KB facts or no further inference can be made



Logic programming using BC

- Main features
  - √ Restricted form of first-order logic
  - √ Mixes control information with declarative sentences
  - √ Backward chaining search: Prove a goal
- *Prolog* is the dominant logic programming language, and has been used for
  - √ Expert systems
  - √ Natural language systems
  - √ Compilers
  - √ Many others

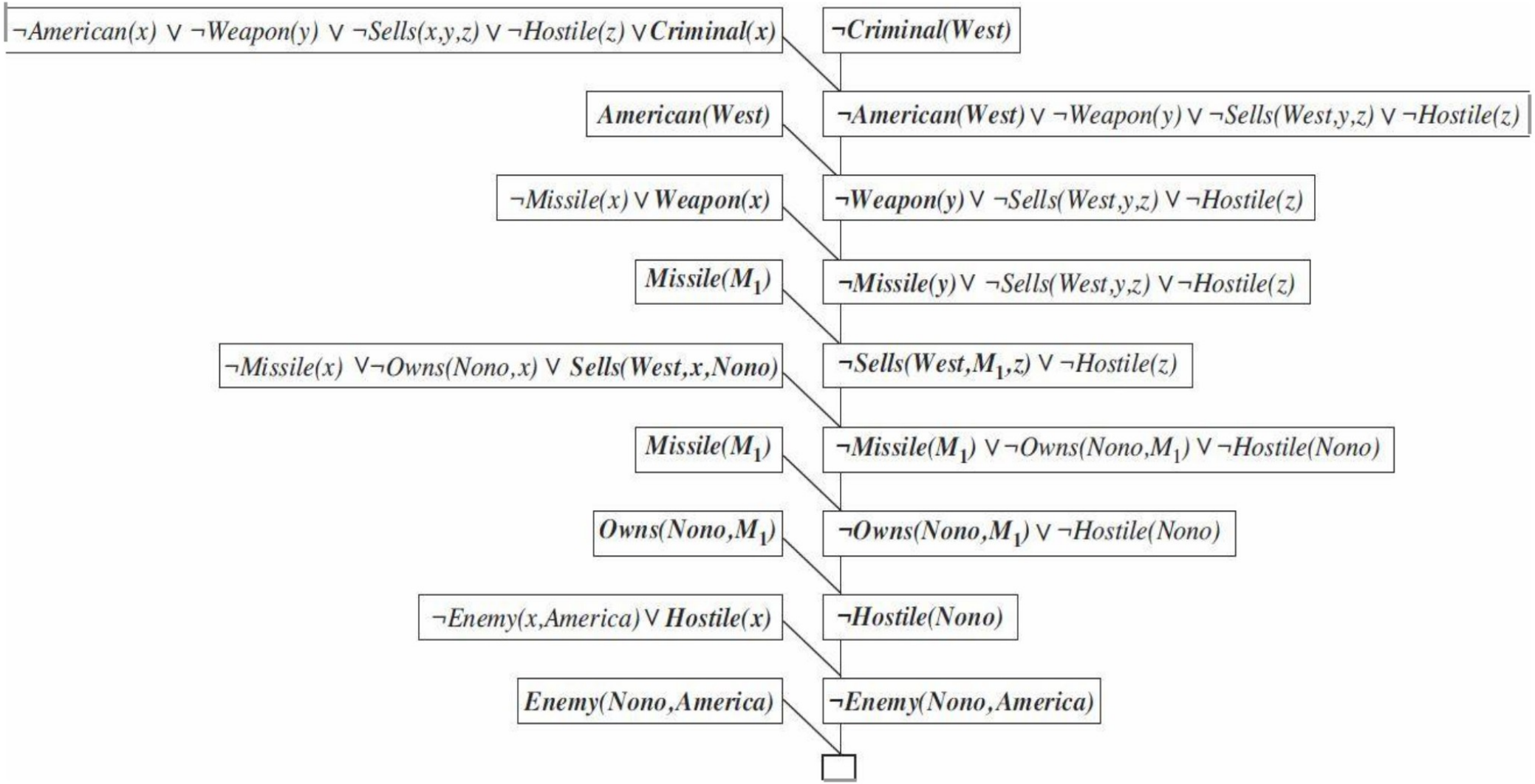
Resolution - A complete inference procedure

- Extends generalized modus ponens by allowing rules with disjunctive conclusions

$$p_1 \wedge p_2 \wedge \Lambda \wedge p_n \Rightarrow q_1 \vee q_2 \vee \Lambda \vee q_m$$

- Resolution rule: $\dfrac{p \vee q, \neg p \vee r}{q \vee r}$
- Assumes that all sentences are written in *a normal form*, for which there exists an efficient algorithm
  - √ E.g. eliminate implications, replace $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$
- A complete resolution inference procedure
  - √ Works by *refutation*, i.e. include $\neg$*Goal* in the KB
  - √ Then apply resolution rule until a contradiction is found

Example resolution proof



Theorem provers using resolution

- Main features
  - √ Allows expression of problems in full first-order logic
  - √ May permit additional control information
  - √ Performs inference by resolution or other procedure
- Applications
  - √ Assistants for mathematicians
  - √ Synthesis and verification of hardware/software systems
  - √ Diagnosis of technical systems

Summary

- Different logics make different *commitments* about the world and what can be known
- *First-order logic* commits to the existence of objects and relations between objects
- An agent can use first-order logic for reasoning about world *states* and for deciding on *actions*
- First-order sentences are made of *terms*, *predicates*, *functions*, *quantifiers* and *connectives*
- First-order logic is *complete* but semi-decidable, but extensions are incomplete

Summary (cont.)

- Reduce to propositional inference by replacing variables with constants - Inefficient
- Generalized Modus Ponens (GMP) relies on *unification*, i.e. partial instantiation of variables
- *Forward chaining* uses GMP to derive new facts from known facts
- *Backward chaining* uses GMP to prove goals from known facts
- *Resolution* is a complete and efficient proof system for first-order logic
- Reasoning systems include *theorem provers*, *logic programming*, and *production systems*

More Details

CNF for FOL

- Every sentence of first-order logic can be converted into an inferentially equivalent sentence in conjunctive normal form (CNF)

CNF for FOL

- Eliminate implications
- Move negation inwards
- Standardize variables apart
- Skolemize existential variables
- Drop universal variables
- Distribute $\vee$ over $\wedge$

Example

- ∀ x (Person(x) ⇒ ¬∀ y (¬Loves(x,y)∨¬Person(y))
- ∀ x (¬Person(x) ∨ ¬∀ y (¬Loves(x,y)∨¬Person(y))
- ∀ x (¬Person(x) ∨ ∃ y ¬ (¬Loves(x,y)∨¬Person(y))
- ∀ x (¬Person(x) ∨ ∃ y (¬¬ Loves(x,y)∧ ¬¬ Person(y))
- ∀ x(¬Person(x) ∨ ∃ y (Loves(x,y)∧Person(y))
- ∀ x (¬Person(x) ∨ (Loves(x,f(x))∧Person(f(x)))
- (¬Person(x) ∨ (Loves(x,f(x))∧Person(f(x)))
- (¬Person(x) ∨ Loves(x,f(x))) ∧ (¬Person(x) ∨ Person(f(x)))

CNF for FOL

- Every sentence of first-order logic can be converted into an inferentially equivalent sentence in conjunctive normal form (CNF)
  - Not logically equivalent
    - Why? Skolemization
  - Inferentially equivalent: (un)satisfiable iff original sentence is (un)satisfiable

FOL Resolution

$$\frac{l_1 \vee \cdots \vee l_i \vee \cdots \vee l_k, \quad m_1 \vee \cdots \vee m_j \vee \cdots \vee m_n}{Subst(\Theta, l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \cdots \vee l_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \cdots \vee m_n)}$$

$$\Theta = Unify(l_i, \neg m_j)$$

Proof by Resolution

- Convert KB to CNF
- Convert ¬α to CNF and add to KB
- Apply resolution rule to complementary clauses (with unification)
- Until you derive the empty clause

Unification

$$Unify(p, q) = \Theta \text{ where } Subst(\Theta, p) = Subst(\Theta, q)$$

- Knows(John,x) and Knows(John,Jane)
  - { x/Jane }
- Knows(John,x) and Knows(y, Bill)
  - {x/Bill, y/John }
- Knows(John,x) and Knows(y, Mother(y))
  - { y/John, x/Mother(John) }
- Knows(John,x) and Knows(x,Elizabeth)
  - Fails

Unification

$Unify(p, q) = \Theta$ where $Subst(\Theta, p) = Subst(\Theta, q)$

- Variables need to be standardized apart
- Occurs check
- Most general unifier: places the fewest restrictions on the variables
  - Is unique
  - AIMA Fig 9.1

FOL Resolution

$$\frac{l_1 \vee \cdots \vee l_i \vee \cdots \vee l_k, \quad m_1 \vee \cdots \vee m_j \vee \cdots \vee m_n}{Subst(\Theta, l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \cdots \vee l_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \cdots \vee m_n)}$$

$$\Theta = Unify(l_i, \neg m_j)$$

Proof by Resolution

- Convert KB to CNF
- Convert ¬α to CNF and add to KB
- Apply resolution rule to complementary clauses (with unification)
- Until you derive the empty clause

The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all its missiles were sold to it by Colonel West, who is American.

To prove: West is a criminal

It is a crime for an American to sell weapons to hostile nations.

$$\forall x, y, z\ American(x) \land Weapon(y) \land Sells(x, y, z) \land Hostile(z) \Rightarrow Criminal(x)$$

Nono has some missiles.

$$\exists x\ Owns(Nono, x) \land Missile(x)$$

All Nono's missiles were sold to it by Colonel West.

$$\forall x\ Missile(x) \land Owns(Nono, x) \Rightarrow Sells(West, x, Nono)$$

Missiles are weapons.

$$\forall x\ Missile(x) \Rightarrow Weapon(x)$$

An enemy of America counts as "hostile."

$$\forall x\ Enemy(x, America) \Rightarrow Hostile(x)$$

West is an American.

$$American(West)$$

Nono is an enemy of America.

$$Enemy(Nono, America)$$

$\forall x, y, z\ American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)$

$\exists x\ Owns(Nono, x) \wedge Missile(x)$

$\forall x\ Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(West, x, Nono)$

$\forall x\ Missile(x) \Rightarrow Weapon(x)$

$\forall x\ Enemy(x, America) \Rightarrow Hostile(x)$

$American(West)$ $Enemy(Nono, America)$

Proof by Resolution

- Convert KB to CNF
- Convert ¬α to CNF and add to KB
- Apply resolution rule to complementary clauses (with unification)
- Until you derive the empty clause

Convert to CNF

- Eliminate implications
- Move negation inwards
- Standardize variables apart
- Skolemize existential variables
- Drop universal variables
- Distribute $\vee$ over $\wedge$

$\forall x, y, z\ American(x) \land Weapon(y) \land Sells(x, y, z) \land Hostile(z) \Rightarrow Criminal(x)$

Eliminate implications:

$\forall x, y, z\ \neg[American(x) \land Weapon(y) \land Sells(x, y, z) \land Hostile(z)] \lor Criminal(x)$

Move negation inwards (DeMorgan's Laws)

$\forall x, y, z\ [\neg American(x) \lor \neg Weapon(y) \lor \neg Sells(x, y, z) \lor \neg Hostile(z)] \lor Criminal(x)$

Standardize variables apart

Skolemize existential variables

Drop universal quantifiers

$\neg American(x) \lor \neg Weapon(y) \lor \neg Sells(x, y, z) \lor \neg Hostile(z) \lor Criminal(x)$

Distribute $\lor$ over $\land$

$$\forall x, y, z\ American(x) \land Weapon(y) \land Sells(x, y, z) \land Hostile(z) \Rightarrow Criminal(x)$$

$$\neg American(x) \lor \neg Weapon(y) \lor \neg Sells(x, y, z) \lor \neg Hostile(z) \lor Criminal(x)$$

$\forall x, y, z\ American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)$

$\neg American(x) \vee \neg Weapon(y) \vee \neg Sells(x, y, z) \vee \neg Hostile(z) \vee Criminal(x)$

$\exists x\ Owns(Nono, x) \wedge Missile(x)$

$Owns(Nono, M_1), Missile(M_1)$

$\forall x\ Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(West, x, Nono)$

$\neg Missile(x) \vee \neg Owns(Nono, x) \vee Sells(West, x, Nono)$

$\forall x\ Missile(x) \Rightarrow Weapon(x)$

$\neg Missile(x) \vee Weapon(x)$

$\forall x\ Enemy(x, America) \Rightarrow Hostile(x)$

$\neg Enemy(x, America) \vee Hostile(x)$

$American(West)$

$American(West)$

$Enemy(Nono, America)$

$Enemy(Nono, America)$

$$\neg American(x) \vee \neg Weapon(y) \vee \neg Sells(x, y, z) \vee \neg Hostile(z) \vee Criminal(x)$$

$$Owns(Nono, M_1), Missile(M_1)$$

$$\neg Missile(x) \vee \neg Owns(Nono, x) \vee Sells(West, x, Nono)$$

$$\neg Missile(x) \vee Weapon(x)$$

$$\neg Enemy(x, America) \vee Hostile(x)$$

$$American(West)$$

$$Enemy(Nono, America)$$

Proof by Resolution

- Convert KB to CNF
- Convert ¬α to CNF and add to KB
- Apply resolution rule to complementary clauses (with unification)
- Until you derive the empty clause

"West is a criminal"

$$Criminal(West)$$

Negate and convert to CNF

$$\neg Criminal(West)$$

$$\neg American(x) \vee \neg Weapon(y) \vee \neg Sells(x, y, z) \vee \neg Hostile(z) \vee Criminal(x)$$

$$Owns(Nono, M_1), Missile(M_1)$$

$$\neg Missile(x) \vee \neg Owns(Nono, x) \vee Sells(West, x, Nono)$$

$$\neg Missile(x) \vee Weapon(x)$$

$$\neg Enemy(x, America) \vee Hostile(x)$$

$$American(West)$$

$$Enemy(Nono, America)$$

$$\neg Criminal(West)$$

Proof by Resolution

- Convert KB to CNF
- Convert ¬α to CNF and add to KB
- Apply resolution rule to complementary clauses (with unification)
- Until you derive the empty clause

$$\neg American(x) \vee \neg Weapon(y) \vee \neg Sells(x, y, z) \vee \neg Hostile(z) \vee Criminal(x)$$

$$Owns(Nono, M_1), Missile(M_1)$$

$$\neg Missile(x) \vee \neg Owns(Nono, x) \vee Sells(West, x, Nono)$$

$$\neg Missile(x) \vee Weapon(x)$$

$$\neg Enemy(x, America) \vee Hostile(x)$$

$$American(West)$$

$$Enemy(Nono, America)$$

$$\neg Criminal(West)$$

$\neg American(x) \vee \neg Weapon(y) \vee \neg Sells(x, y, z) \vee \neg Hostile(z) \vee Criminal(x)$

$Owns(Nono, M_1), Missile(M_1)$

$\neg Missile(x) \vee \neg Owns(Nono, x) \vee Sells(West, x, Nono)$

$\neg Missile(x) \vee Weapon(x)$

$\neg Enemy(x, America) \vee Hostile(x)$

$American(West)$

$Enemy(Nono, America)$

$\neg Criminal(West)$

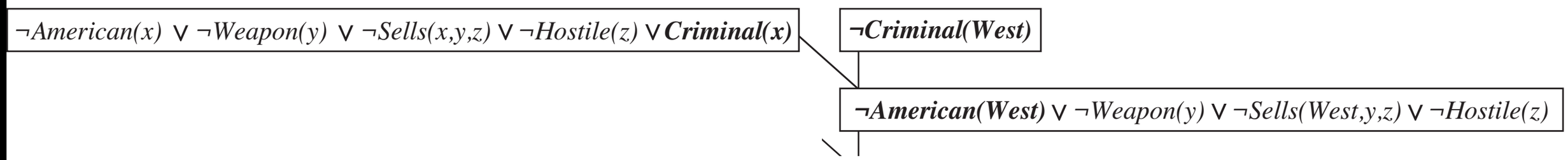
{ x/West }

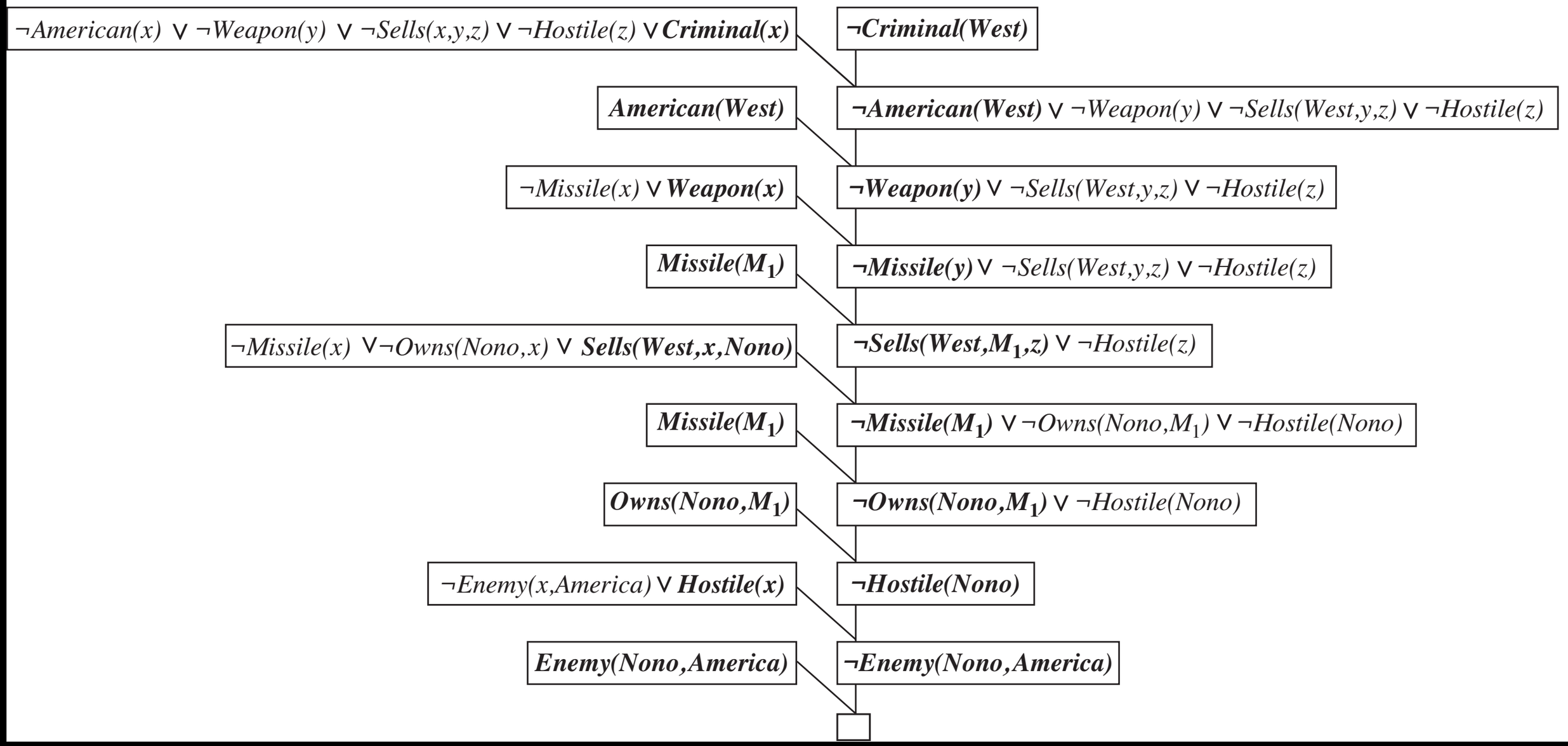
$$\neg American(x) \vee \neg Weapon(y) \vee \neg Sells(x, y, z) \vee \neg Hostile(z) \vee Criminal(x)$$

$$\neg Criminal(West)$$

{ x/West }

$$\neg American(West) \vee \neg Weapon(y) \vee \neg Sells(West, y, z) \vee \neg Hostile(z)$$


¬American(x) ∨ ¬Weapon(y) ∨ ¬Sells(x,y,z) ∨ ¬Hostile(z) ∨ Criminal(x)
¬Criminal(West)
¬American(West) ∨ ¬Weapon(y) ∨ ¬Sells(West,y,z) ∨ ¬Hostile(z)


¬American(x) ∨ ¬Weapon(y) ∨ ¬Sells(x,y,z) ∨ ¬Hostile(z) ∨ Criminal(x)
¬Criminal(West)
American(West)
¬American(West) ∨ ¬Weapon(y) ∨ ¬Sells(West,y,z) ∨ ¬Hostile(z)
¬Missile(x) ∨ Weapon(x)
¬Weapon(y) ∨ ¬Sells(West,y,z) ∨ ¬Hostile(z)
Missile(M1)
¬Missile(y) ∨ ¬Sells(West,y,z) ∨ ¬Hostile(z)
¬Missile(x) ∨¬Owns(Nono,x) ∨ Sells(West,x,Nono)
¬Sells(West,M1,z) ∨ ¬Hostile(z)
Missile(M1)
¬Missile(M1) ∨ ¬Owns(Nono,M1) ∨ ¬Hostile(Nono)
Owns(Nono,M1)
¬Owns(Nono,M1) ∨ ¬Hostile(Nono)
¬Enemy(x,America) ∨ Hostile(x)
¬Hostile(Nono)
Enemy(Nono,America)
¬Enemy(Nono,America)

Resolution Proof

Since $KB \cup \{\neg Criminal(West)\}$ is unsatisfiable,

$$KB \models Criminal(West)$$

Special Case: Forward Chaining

- Knowledge base of definite clauses
  - Implications with single literal conclusion
- Starting from known facts, trigger rules whose premises are satisfied
  - Using substitution to match
- Add their conclusions to the KB

Special Case: Backward Chaining

- Work backward from the goal, chaining through rules to find known facts that support the proof
  - Allow substitutions when matching facts and rules
- DFS => incomplete
- The basis of logic programming (Prolog)

FOL Inference

- Semantics of first-order sentences
- Entailment: same as PL!
- Lifted inference rules
- Resolution (first-order CNF, unification)
  - Proof by contradiction
- Forward and backward chaining